Understanding Optimization Landscapes using Bio-inspired Algorithms

Project 3 IT3708 – Spring 2025

Groups allowed: Groups are allowed and encouraged, up to 3 students. You can keep the same group from previous projects. All students in the group must be present during the demo day. All members of a group should sign up for the same time slot on demo day. For this project, you will do a conference-style presentation.

Guidance: Questions will be answered during lab hours or on Blackboard's discussion board.

Deadline: 27.04.2026

Introduction

The last two projects of this course have dealt with solving problems *practically*, that is, finding a specific solution that is close to the optimum. However, finding solutions is not all we do as professionals. Multiple times we need to create algorithms ourselves or compare different solution methods. Sometimes we even need to find new problems to tackle. This project will let you practice these skills while looking at a familiar problem with a different, more theoretical mindset.

Lab Goals

- Read and parse large evolutionary landscapes based on feature selection, as well as implementing a synthetic landscape
- Create a visualization of the fitness landscape to highlight optima and get a general sense of the complexity of your fitness function
- Optionally, create a visualization of the evolutionary algorithms behaviour on the search spaces to better understand what is actually happening.
- Implement (or reuse) a **single objective evolutionary algorithm** of your choice
- Implement a **multi-objective evolutionary algorithm** of your choice
- Implement a **swarm intelligence optimization algorithm** of your choice
- Compare your algorithms and show your experimentation results
- Test your algorithms on an *unseen* landscape and try to find **multiple optima**

Problem Statement

You may have heard multiple times about the *No Free Lunch theorem in search and optimization*[1]: There is no such thing as a single best solver for all problems. Solvers do well because they happen to have built in assumptions that align with the structure of the problem. In this project, we will try to capture the structure of the problem and study how different algorithms behave on the problem's terms.

The problem is the same as in Project 1: **feature selection**, which is a machine learning process where redundant and irrelevant features are removed from a dataset [1]. Evolutionary algorithms have been successfully used for the purpose of feature selection [10]. The selected features will often provide approximately as good, or better, results, and having data with fewer dimensions has some advantages when training a predictive model.

However, as you have learned throughout the course, there might be **multiple solutions** to this problem. What if we want to **find them all**? What if the solution that we found is **too complex to explain** to stakeholders? What if there is another combination of features with *good* results but with fewer columns involved?

You think for a moment and decide to study the **generalized feature selection problem**. You no longer want to find the best combination of columns for a particular dataset, you want to try to make arguments about datasets and the performance of models in general. What do different problems look like and what does it mean to navigate them? How do problems change depending on the nature of the datasets? Do some algorithms perform better on certain problems? Do some algorithms always trip up on certain problem structures? What characterizes problems which are easier and harder for different algorithms? After some deliberation, you arrive to a simple (but effective) model for reducing the complexity of the model (what we call *regularization*[2]):

Quality of a subset of features = The ML model performance (good) - How difficult it is to explain (bad)

Or in mathematical notation, using a canonical form with sums [8]:

$$h(x) = h_a(T(x)) - \epsilon h_P(x), \qquad \text{(Eq. 1)}$$

where

- x is a subset of <u>features</u> (combination of columns)
- T is any given machine learning <u>model</u> trained on any given <u>dataset</u> using x
- h_a represents <u>the accuracy</u> (in classification, or prediction, depending on T)
- h_P represents <u>a penalty</u> for the number of columns (or features) used, and
- ϵ represents <u>the weight</u> of the penalty.

Since $h(x)$ is about the accuracy and a penalty, this is something you want to **maximise**
.

[1] https://en.wikipedia.org/wiki/No_free_lunch_in_search_and_optimization
[2] https://en.wikipedia.org/wiki/Regularization_(mathematics)

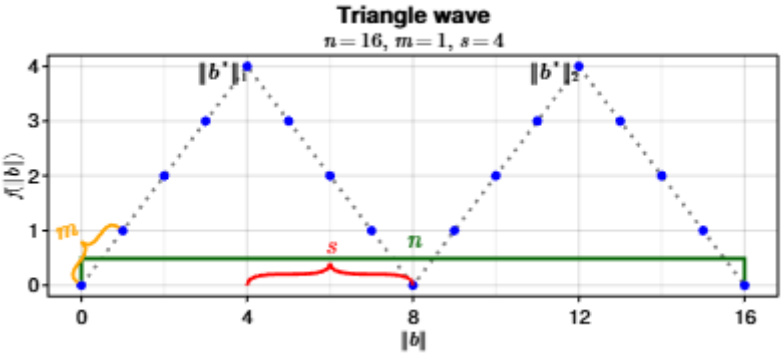
For the Synthetic problem, we ask that you implement the triangle function. The function will be covered in greater detail in the project presentation on March 26th; alternatively, read section 3.2 of [11].
The function (taken from [11]) is defined by these formulas:

$$\textsc{Triangle}(\boldsymbol{b}, m, s) = \begin{cases} g(\boldsymbol{b}), & \text{if } \left\lfloor \frac{\|\boldsymbol{b}\|}{s} \right\rfloor \mod 2 = 1 \\ m\left(\left\lceil \frac{\|\boldsymbol{b}\|}{s} \right\rceil \cdot s - \|\boldsymbol{b}\|\right) & \text{otherwise} \end{cases} \tag{4}$$

where

$$g(\boldsymbol{b}) = \begin{cases} m \cdot s, & \text{if } \|\boldsymbol{b}\| \mod s = 0 \\ m(\|\boldsymbol{b}\| \mod s) & \text{otherwise.} \end{cases} \tag{5}$$

This might seem really complicated, but the below figure hopefully explains it better:



We see that n is the dimensions of the function in use, equivalent to the amounts of active columns of our feature selection datasets. It represents the x axis in the figure, where each point along the x axis represents how many active “columns” are 1’s/active. The y axis is the fitness. m is the increase in fitness per step. Parameter s is the amount of steps taken until you get to the next reversal in trends (increasing or decreasing m). You should use **n** = 16, **m** = 1, and **s** = 4.

More details will be available in the lab lecture.

Step 1—Firstly, your task is reading and parsing the data files provided, then choosing **which penalty** to apply. As well as implementing the function that generates the synthetic landscape.
Potential penalty: using the number of columns, multiplied by a small constant ϵ between 0 and 1.

Project Tasks

Because training these models is expensive and subject to lots of parameters, which can provide quite different landscapes, we supply the required tables as HDF5 files. It is your task to parse these files. There are 2 instances of stored data in the files. For this project the fitness landscape is based on the accuracy section, the times are also included as you could use training time in your penalty function. Each row-index corresponds to the decimal representation of the binary combination of whether a column is used as an active feature during the training of the machine learning algorithm trained. It works similarly to what you used in Project 1. The column corresponds to each instance of this combination of features being used for training the machine learning algorithm. The value in each element is the accuracy of the machine learning algorithm (1-errors). As you can see the same combination

of fitnesses gives varying results in the different columns. You may have noticed the same happening during project 1 if looking up the fitness of a combination multiple times by recalculating it. We would like you to calculate the mean of these columns and use this as the basis of the lookup table before you apply any penalties. Note that the HDF5 files for the feature selection landscapes start on index 1, because training a machine learning algorithm on 0 features doesn't really make much sense. So the total size of the landscapes is 2^n - 1. This is **NOT** the case in the synthetic landscapes.

Step 2—store these lookup tables in your program(s), either applying the penalties directly while creating the tables, or applying them later on lookups.

Programming Language

You can use **any** programming language you want. However, be prepared that the test data of one of the files given will be significantly larger than what you used for the training data, as such, we would highly recommend that you don't use Python. This is true for the fitness function searching as well as the visualization step. Some form of parallelization is highly recommended, while traversing the search space, and when generating the synthetic functions. If using Julia, take a look at *MLJ.jl* and *Distributed.jl*.

Visualization

Your tables represent four different problems, each with its unique search space, and each with its own **fitness landscape**. To start your analysis, you may want to visualize the landscape to help you pick suitable algorithms for your experimentation and understand the landscape better.

The Fitness Landscape[3] is an analogy to understand how complex it could be for any optimization algorithm to efficiently navigate the search space. Normally, one does not have access to the *entire picture* of a landscape, but luckily, you have access to all the precomputed fitness values —one for each combination of columns (or each point in the search space). Using the information provided by your lookup table, you can now make *a map*.

Step 3 —Visualization: Based on the **precomputed training tables**, create a **visualization** for each landscape. There are several ways to do this, and we will cover some of them in our Landscape Analysis lecture. Choose any **visualization method** you like. You are required to highlight each point considered as a **local optimum** in your landscape.

How you define a local optimum is also up to you. If you are using a bitstring, you might want to include only peaks, or you could consider plateaus as well. If you are using a continuous chromosome, you must define a threshold for optimality.

[3] Fitness landscape from 1:30 and onwards: FT018 - Function Landscapes and the Limits of AI for Bio
[4] By Niche we mean a basin of attraction, all the points where local search leads to the same local optima

Visualization of Algorithm Behaviour (optional)

In addition to visualizing the landscape, it is also highly useful to understand how your algorithms actually behave on the given landscapes. This will provide useful information on understanding how the algorithms work, and what affect the parameters and methods used have on the landscape. This is not an easy task to do, as much research focuses on this area, and it would be unreasonable to expect you to highlight every aspect of the algorithm behaviour, especially given the high dimensionality of the fitness landscape making simple graph visualizations difficult. As such we would like you to highlight at least three key aspects of the algorithm behaviour. Be creative for this one, however, to get some ideas, some examples are:

- Which niches[4] get filled
    - How many individuals reside in each niche
    - How many niches remain empty
    - How does this change over time
    - How do individuals move between niches
- What effect does crossover have on individuals vs mutation on the landscape?
- How does an individual traverse the search space over time
- When in a niche, how quickly does it converge to the local optima if ever
    - How efficient is your algorithm at not getting stuck in local niches
- How does the entropy change over time
- How does crowding work within a niche
    - Does the Crowding algorithm force neighbors to compete when they are in different niches
- Highlight key differences between the different algorithms on any of the points above
- Highlighting differences in the distribution of individuals with different focus on the different objectives in multi-objective optimization.

This may seem like a lot, but you can easily visualize several of the aspects in the same figure. You also don't need to do this for every instance and every algorithm, as it would take too long to show us, but we want you to see what your algorithms do, what limitations they have, how they compare to each other and similar. You can focus on the entire population on the entire fitness landscape, an individual's path on the landscape, only highlighting a single niche or similar, as long as it shows some key aspect of how the algorithms works.

Step 4 (Optional)—Visualization of algorithm behavior: based on the **precomputed tables**, create a **visualization** that shows 3 key aspects of how the algorithm(s) works on one or multiple landscapes. There are several ways to do this, and we will cover some of them in our Landscape Analysis lecture.

Programming Language

Again, you can use **any** programming language you want. There are plenty of professional plotting libraries in Julia, Python, C++, Java, etc. This step is quick and it is only done once,

so you do not need to worry too much about computational speed. Although keep in mind that one of the test datasets will be quite large, so if you wish to show every point in the landscape you ought to stay away from Python. If you are using Julia, *Plots.jl* is a good first choice. If you want to have more control, take a look at *Makie.jl* and *CairoMakie.jl* for a more professional look.

Implementation

When you have the tables it'll be very easy to test as many evolutionary algorithms as you want, because you only need to look for the fitness value in the table. This is very fast, even if the table has a million elements.

Step 4—Implementation: Implement <u>any three algorithms of your choice:</u>

- One **single objective** evolutionary algorithm (for example., the one you used in Project 1)
- One **multi-objective** evolutionary algorithm
- One **swarm intelligence** optimization algorithm (for example, the one in EvoLP.jl)

Remember that your tables represent different fitness functions, so every time your algorithm needs to evaluate an individual, just look at the value in the corresponding table. One example of a selection of algorithms to implement could be:

I want to use the SGA, along with the NSGA-II, and PSO.

Experimentation and Presentation of Your Results

You are now ready to experiment and gather your results. With these results, you can draw conclusions about the algorithms you implemented and their search behavior, the datasets used, and even the machine learning models you trained. Did any of the landscapes prove to be more difficult for a specific algorithm?

You can annotate your results in a table, and create visuals to help convey your claims to your scientist colleagues. At least one of these elements must be presented during your demonstration.

Parameter tuning and control:

A very important note is that there are some restrictions on what parameters you can use for this project. We expect you to **use the SAME parameters** for your given algorithm on all 3 test landscapes. This means you cannot tune to the individual landscapes. You will also **NOT** be allowed to use custom initialization for the start of each landscape, or use exhaustive search/information from optima to trivialise this task. You can use limited memetic algorithms to make a single step towards a local optima or similar, but not exhaustive local searches. This is because this is not how you would ever deal with landscapes in the real world, because you simply wouldn't have the information needed, and you are dependent on the parameters chosen on previous data (the train datasets).

However, to make it achievable, you are allowed 2 techniques:

- You CAN have parameters dependent on constants that are dependent of the problem,
  For example population size that scales by some factor equivalent to the search space (n^2, or something)
- And you can have Parameter control (that is to say you can change the parameters during the run), adaptive or self adaptive would be particularly useful. For example increasing mutation rate when the diversity is too low.

Step 5—Experimentation and Comparison: Run your three algorithms on each landscape, and gather your results in a **table** or **plot**. Then, write your **conclusions** about them. What are the **hyper-parameters** you used to find the global optima for each algorithm? Can the algorithms find **multiple optima**? Did you **control the parameters** during the run?

To make a fair comparison, you need to run the algorithms several times using similar hyperparameters. This is to eliminate any lucky initializations and see if the behavior observed is actually consistent. You are required to present statistical measures along with your results to ensure a fair comparison. For example:

I will run my algorithms 10 times each, store the results in a table, and calculate the average fitness value of the best performing individual across the runs. I will then create a table to present the results, along with the standard deviation of my sample of runs.

Testing your Algorithms on Unseen Instances

By playing around with your algorithms on different landscapes, you might now have an intuition on how they move through the search space. You were navigating each landscape with the lights on. This is not usually the case, as such we don't want you using this information for your algorithms

A few days before the demo, we will provide you with three **new problems** that you need to navigate. 2 of these will be similar as the ones you are used to from before, as in they are also feature selection landscapes. For these your task will then be to find the optima. You will be provided with a specific regularization constant for these problems. The question you need to answer is what are the subsets of features that minimize $h(x)$?

The last landscape however is different. It will be a synthetic problem designed by us, based on some existing approaches from the literature. We will explain exactly how it is generated, and the underlying methods for why it looks the way it does. This problem will be **MUCH** bigger than the previous landscapes provided, and way more difficult. You are not expected to solve this problem, but we want you to understand the theory that explains your algorithm behaviour. More details will be provided with the dataset. You are expected to run your algorithm on this instance many times, and make a table of the results. (Hint: Doing visualization of algorithm behaviour for this task makes it much easier). If you do manage to solve it consistently, you are expected to explain **how.**

Step 6—Test Instances: Run <u>any</u> of your three algorithms on each of the test instances, and try to find the global optima. It is **very important** that you specify not only the optimal value $h(x^*)$, but also the point in the search space it corresponds to (the optimum, x^*).

Your grade will be based on *how close* you are from any of the global optima, which will be "unknown" to you.

For the third dataset you are not given points for your solution, but still expected to provide the above information. You are instead graded on understanding ***WHY*** your algorithm(s) fail.More details will be released when you get the datasets.

Evaluation

As in project 2, you can get a **total of 25 points for this assignment (plus 3 extra points[5])**:

- Report and presentation of your work (10 points)
- Project tasks (15 points)

Project Tasks

The project tasks have the following weights:

- Step 1: Parse lookup table data - 0 points
- Step 2: Storing and penalizing the fitness function - 0 points
- Step 3: Visualization – 2 points
- Step 3 (Optional): Visualization of Algorithm behaviour– +2 points
- Step 4: Implementation – 5 points (1.66 points for each algorithm)
- Step 5: Experimentation and Comparison – 6 points
- Step 6: instances - up to 3 points, one for each task

[5] Only applicable for this project

Delivery Method and Deadline

You should hand in **2 deliverables in Blackboard** (a report, and your code), and give a **brief presentation** (10 minutes) during the **demo day**:

- **Code**: you should deliver a zip file with your code on Blackboard
- **Report**: you should deliver a group report (no more than 10 pages: 6 pages of the report and maximum 4 pages of appendix), summarizing the methodology, your experimental setup, your visualizations, the results of your experiments, your findings and your conclusions. Remember to include your references (including the datasets) and **cite them properly**.

The submission system will be closed on **April 27th 2026**. The **demo day** for Project 3 is also **April 27th 2026**. A signup schedule will be announced a week before. Please follow Blackboard for details. **Every student must submit a copy of their jointly developed code and report. You must attend the demo individually on the scheduled demo date. No early or late submission or demo will be entertained except in case of an emergency.**

Additional Reading on Landscape Analysis

We highly recommend taking a look at our paper on regularized feature selection, presented at Parallel Problem Solving from Nature '24 [8]. In the same conference, other researchers presented very similar results using different visualization methods [3].

If you want to know more about Landscape analysis in general, we highly recommend the Workshop from Gabriela Ochoa and Katherine Malan at GECCO '24 [5]. And, in case you want to delve deeper into visualization methods, you can read our PPSN '24 paper, or take a look at more expensive but highly descriptive methods like Local Optima Networks [4, 6]

Plenty of examples of visualization will also be provided in the project lecture. As such we highly recommend you attend the lecture or at least look over the slides.

References

[1] Isabelle Guyon and André Elisseeff. 2003. An introduction to variable and feature selection. *J Mach Learn Res* 3, null (March 2003), 1157–1182.

[2] Kelly Markelle, Rachel Longjohn, and Kolby Nottingham. UCI Machine Learning Repository. *UCI Machine Learning Repository*. Retrieved January 29, 2024 from https://archive.ics.uci.edu/

[3] Arnaud Liefooghe, Ryoji Tanabe, and Sébastien Verel. 2024. Contrasting the Landscapes of Feature Selection Under Different Machine Learning Models. In *Parallel Problem Solving from Nature – PPSN XVIII*, 2024. Springer Nature Switzerland, Cham, 360–376. https://doi.org/10.1007/978-3-031-70055-2_22

[4] Werner Mostert, Katherine M. Malan, Gabriela Ochoa, and Andries P. Engelbrecht. 2019. Insights into the Feature Selection Problem Using Local Optima Networks. In *Evolutionary Computation in Combinatorial Optimization* (*Lecture Notes in Computer*

Science), 2019. Springer International Publishing, Cham, 147–162. https://doi.org/10.1007/978-3-030-16711-0_10

[5] Gabriela Ochoa and Katherine Malan. 2024. Landscape Analysis of Optimisation Problems and Algorithms. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion* (*GECCO '24 Companion*), August 01, 2024. Association for Computing Machinery, New York, NY, USA, 984–1004. https://doi.org/10.1145/3638530.3648421

[6] Gabriela Ochoa, Sébastien Verel, Fabio Daolio, and Marco Tomassini. 2014. Local Optima Networks: A New Model of Combinatorial Fitness Landscapes. In *Recent Advances in the Theory and Application of Fitness Landscapes*, Hendrik Richter and Andries Engelbrecht (eds.). Springer, Berlin, Heidelberg, 233–262. https://doi.org/10.1007/978-3-642-41888-4_9

[7] Xavier Fernando Cuauhtémoc Sánchez Diaz. 2024. Replication Data for: Regularized Feature Selection Landscapes: An Empirical Study of Multimodality. https://doi.org/10.18710/DQZKMX

[8] Xavier F. C. Sánchez-Díaz, Corentin Masson, and Ole Jakob Mengshoel. 2024. Regularized Feature Selection Landscapes: An Empirical Study of Multimodality. In *Parallel Problem Solving from Nature – PPSN XVIII*, September 07, 2024. Springer Nature Switzerland, Cham, 409–426. https://doi.org/10.1007/978-3-031-70055-2_25

[9] Joaquin Vanschoren, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. 2014. OpenML: networked science in machine learning. *ACM SIGKDD Explor. Newsl.* 15, 2 (June 2014), 49–60. https://doi.org/10.1145/2641190.2641198

[10] Bing Xue, Mengjie Zhang, Will N. Browne, and Xin Yao. 2016. A Survey on Evolutionary Computation Approaches to Feature Selection. *IEEE Trans. Evol. Comput.* 20, 4 (August 2016), 606–626. https://doi.org/10.1109/TEVC.2015.2504420

[11] Sánchez-Díaz, X., Mengshoel, O.: Estimating the number of local optima in multimodal pseudo-boolean functions: Validation via landscapes of triangles. In: Press. Proceedings of the Genetic and Evolutionary Computation Conference Companion, GECCO 2024 (Jul 2024)